

Beyond Task Completion: A Verification-vs.-Conformance Gap in Tool-Evolving Agents

Anonymous Author(s)

Abstract

Agents that synthesize their own tools ship a second artifact alongside each answer: a software library that future tasks reuse, compose, and depend on. Task completion (TC) certifies the answer; it does not certify the library. We show that this matters. On a Claude Haiku 4.5 verified-subset pilot, we patch the harness to preserve per-tool source and replay every synthesised tool against a held-out conformance suite for the capability it was built for. Across 222 preserved tools and three tool-creating protocols, 96.8% (215 of 222) record per-tool correctness $C=0.00$: two protocols silent-rot at 100%, one at 91.7%. The suites are not the problem: hand-written reference implementations released with the benchmark score $C=1.00$ on all 16 capability suites. The synthesised tools execute cleanly and pass in-session verifier calls – they simply return wrong answers on inputs the agent never saw. This is the verification-vs.-conformance gap a pass-rate benchmark cannot see.

EVOLVETOOL-BENCH makes that gap measurable. Sessions are structured into seed, gap, variant, composition, regression, and adversarial roles; runs emit per-tool manifests, verified-subset TC, correct-vs.-incorrect reuse, and audit traces designed to be re-emitted as a post-deployment monitoring schema. In the verified-subset Haiku pilot over five protocols (3 seeds, 8 sessions, 51 verified decisions per pass), TC alone does not separate the protocols; the audit layer does. One pre-specified contrast shows the adapted decision-split synthesis protocol *underperforming* unvalidated one-shot synthesis (−7.1 pp, nominal 95% CI [−13.9, −0.1]; not significant after the pre-registered BH correction) – a hypothesis-generating signal, not a confirmed effect.

We release the harness, per-tool source preservation, conformance replay, verifier definitions, and reproducibility manifests, and argue that an audit schema of this shape – verifier coverage, reuse decomposition, held-out conformance – is the minimum a deployed tool-evolving agent should emit between releases.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → **Software verification and validation**.

Keywords

agentic AI, tool use, evaluation, benchmark, auditability, verification

1 Introduction

A growing class of agents does more than call a fixed tool set. These systems write Python functions, API clients, parsers,

analysis scripts, or richer skill bundles, and then carry those artifacts forward into later tasks. VOYAGER learns reusable code skills in Minecraft [Wang et al.(2023)]; CREATOR separates tool creation from tool use [Qian et al.(2023)]; LLMs as Tool Makers and ToolMaker study agents that construct tools for other agents to call [Cai et al.(2023), Wölfllein et al.(2025)]; EvoSkill and CoEvoSkills treat skills as evolving units of agent knowledge [Alzubi et al.(2026), Zhang et al.(2026)]. In all of these settings, the agent leaves behind a software artifact.

A task-completion score tells us whether the answer was accepted; it does not tell us what the agent left behind. A generated library can solve the next task while still containing brittle parsing logic, redundant functions, or behavior changes that silently break earlier capabilities. For deployed or long-running agents, those differences are not cosmetic: they are the difference between a one-off success and an auditable system.

EVOLVETOOL-BENCH is built around this distinction. It evaluates agents that may create tools during a session and asks not only “did the task pass?” but also “what happened to the library?” Each session has known dependency structure: seed tasks test provided tools, gap tasks invite new capabilities, variant tasks test reuse, composition tasks test chaining, regression tasks revisit prior behavior, and adversarial tasks probe edge cases. The resulting traces let a researcher inspect whether a failure came from synthesis, reuse, verification, composition, or regression.

The benchmark design contains 99 tasks across three domains. Of these, the data-transformation and numerical domains currently expose deterministic task verifiers; the API-orchestration session is part of the benchmark design but excluded from the main TC denominator until its verifiers are complete. Unverified tasks are reported separately and not credited as successes. This narrows the empirical claim of any single submission while tightening the evaluation claim of the framework as a whole.

Our contributions are:

- (1) **A diagnostic protocol for evolving tool libraries.** Sessions are structured so that creation, reuse, composition, regression, and adversarial behavior can be inspected separately.
- (2) **Artifact-level measurements beyond TC.** We define and log tool creation, raw reuse, correct reuse, incorrect reuse, utilization, composition, and regression signals alongside verified task completion.
- (3) **Verifier-aware reporting.** The strict analysis separates the benchmark’s full design scope from the currently verified subset, preventing unverified tasks from inflating task-completion claims.

- (4) **An audit-trace view of agent evaluation.** Runs preserve task outcomes, tools created, tools used, reuse outcomes, and verifier status so that aggregate scores can be traced back to concrete artifacts.

The headline empirical finding is twofold (§5). First, TC alone fails to separate the protocols on the verified-subset Haiku pilot; the only contrast whose nominal CI excludes zero runs in the unexpected direction (the adapted decision-split protocol trails one-shot synthesis) and does not survive multiplicity adjustment. Second, an in-regime conformance probe across all three tool-creating protocols (222 preserved tools) shows the verification-vs.-conformance gap directly: 96.8% of preserved tools record $C=0.00$ on the held-out conformance suite for the capability they were synthesised for, even while session-level TC in the runs that produced those tools sits between 0.31 and 0.34. Together these argue that benchmarks for tool-evolving agents must make the state of the tool library visible – not only the pass rate.

2 Related Work

Tool and skill creation. Systems such as CREATOR, LLMs as Tool Makers, ToolMaker, EvoSkill, and CoEvoSkills all study agents that create tools or skills rather than merely selecting from a fixed tool set [Qian et al.(2023), Cai et al.(2023), Wölflein et al.(2025), Alzubi et al.(2026), Zhang et al.(2026)]. Their evaluations typically focus on downstream success, per-tool validation, or skill-level pass rates. EVOLVE TOOL-BENCH is complementary: it asks what happens to the library over time, especially under reuse, composition, and regression pressure.

Agent and tool-use benchmarks. ToolBench and AgentBench evaluate tool-using agents with predefined environments or APIs [Qin et al.(2023), Liu et al.(2024)]. VendingBench stresses long-horizon agent behavior with a fixed operational environment [Backlund and Petersson(2025)]. These benchmarks are valuable, but the tool inventory is generally not an artifact produced by the agent. Our setting differs because the generated library is part of the system under evaluation.

Skill benchmarks and library learning. SkillsBench shows that skills can help some tasks and hurt others, especially when the skills are self-generated [Li et al.(2026)]. Program-synthesis work such as DreamCoder studies learned libraries and evaluates their effect on solve rates or compression [Ellis et al.(2021)]. The question we address sits between these traditions: when an LLM agent accumulates executable artifacts, can we tell whether the resulting library is correct, reusable, and stable?

Quality of generated code. Tool-Genesis, NoFunEval, and related code-evaluation work measure compliance, unit-test performance, maintainability, or non-functional requirements for generated code [Xia et al.(2026), Singhal et al.(2024)]. EVOLVE TOOL-BENCH borrows the software-engineering lens but applies it at session and library granularity. A single

Table 1: Task roles in each session. The fixed order lets the harness attribute behavior to creation, reuse, composition, regression, or edge-case handling rather than reporting one undifferentiated pass rate.

Role	Count	Diagnostic purpose
Seed	3	Use provided tools; establish baseline behavior.
Gap	2	Solve a task requiring a missing capability.
Variant	2	Recognize that a prior capability can be reused.
Compose	1	Chain multiple capabilities or tools.
Regress	1	Revisit an earlier contract after the library has changed.
Adversarial	2	Handle edge cases or malformed inputs.

generated function can look reasonable in isolation while still making the library harder to use.

3 Benchmark Design

3.1 Sessions and task roles

A session is a fixed sequence of 11 tasks with shared library state. The sequence is intentionally simple: it creates controlled opportunities for a system to use, create, reuse, compose, and preserve tools. Table 1 summarizes the task roles.

The benchmark contains three domains and nine sessions: five data-transformation sessions, one API-orchestration session, and three numerical-computation sessions. Data-transformation sessions use proprietary formats such as binary encodings and schema-like validators. Numerical sessions use curve fitting, signal processing, and constrained optimization tasks. The API session uses a mock service with HMAC authentication and cursor handling. The full benchmark contains 99 tasks, but the strict analysis in this paper uses only tasks with deterministic verifiers.

3.2 Verification policy

Verification is part of the contribution, not a footnote. A task contributes to strict TC only if it has an explicit expected output or a custom verifier; tasks without deterministic verification are reported separately, never credited because the answer was long or plausible. This avoids the common failure mode in agent benchmarks where format-looking output is mistaken for correctness.

The API-orchestration session is excluded from the main strict TC analysis because its task verifiers are not yet complete. The verified-subset pilot therefore covers 8 sessions and 51 verified task decisions per complete system pass. The API session remains in the benchmark design as an important tool-use domain, but is not used to support the numerical claims in this submission.

Table 2: Verifier coverage of the strict subset by task role (eight verified-subset sessions; API-orchestration excluded). Seed tasks deliberately carry no graded outcome and are not part of the TC denominator. The roles that the diagnostic framework leans on – gap, variant, regress – are near-fully covered; compose and adversarial have partial coverage.

Role	Verified	Total	Coverage
Seed (warm-up, ungraded)	0	24	—
Gap	15	16	93.8%
Variant	15	16	93.8%
Compose	4	8	50.0%
Regress	8	8	100.0%
Adversarial	9	16	56.2%
Strict-subset total (graded)	51	64	79.7%

Table 2 shows how the 51-task verified subset is distributed across task roles. Seed tasks carry no graded outcome by design: they exist to expose the agent to the provided library before any creation, so they have no expected output or verifier. The diagnostic-critical roles – gap, variant, regress – are near-fully covered (93.8–100%); compose and adversarial are partially covered. The verified subset therefore exercises the roles the framework leans on for its claims, while remaining honest about where verifier work is still needed.

3.3 Metrics and audit trace

Verified task completion (TC). TC is the fraction of verified tasks whose outputs satisfy the task verifier. It measures user-visible success on the verified subset: necessary but not sufficient.

Tool creation and reuse. Before each task, the harness records the current library; after the task, it records newly created tools and tools used. Reuse is counted when a task invokes a tool that existed before the task began. We decompose reuse into:

- **correct reuse:** a prior tool was used and the task passed;
- **incorrect reuse:** a prior tool was used and the task failed.

We report the *correct-reuse rate* as the fraction of reuse calls that coincide with a passing task; we avoid the term “precision” because no false-positive denominator is defined. This distinction is important because raw reuse can mean either useful abstraction or repeated reliance on a defective artifact.

Library-health diagnostics. The broader framework also supports redundancy, utilization, composition, regression stability, and capability-aligned hidden conformance tests. These diagnostics are part of the benchmark design and audit schema. The verified-subset submission reports only the signals aligned with the current verified manifests: TC, tools created, correct reuse, incorrect reuse, and the correct-reuse rate. A composite library-health ranking is held back until the

```
{
  "task_id": "data_transform.s1/gap.1",
  "role": "gap", "verifier_status": "verified",
  "model": "claude-haiku-4-5", "seed": 0,
  "outcome": "pass",
  "tools_available_before": ["parse_header", ...],
  "tools_used": ["decode_abr_format"],
  "tools_created": ["decode_abr_format"],
  "reuse_outcome": null,
  "artifact_summary": {"loc": 47, "imports": ["base64"]},
  "llm_calls": 3
}
```

Figure 1: One audit-trace record (schema, redacted). The same schema is the unit of post-deployment monitoring discussed in §6.

conformance-test alignment is complete (Supplement §E reports it on a separate Sonnet pilot where source preservation made the alignment possible).

Audit trace. Each run emits a per-task record so aggregate scores can be traced to concrete artifacts. Figure 1 shows the schema for one task record from `data_transform.s1`: a reader can move from any cell in Table 3 to the task, seed, tool, and pass/fail decision that produced it.

4 Experimental Setup

The strict pilot evaluates five representative protocols. They are not meant to exhaust the design space; they provide enough variation to test whether the harness can distinguish tool-creation and reuse behavior.

- (1) **No-Evolution.** The system receives the seed tools and never modifies the library. It is the lower-bound protocol for library evolution.
- (2) **One-Shot.** When the system attempts tool creation, it adds a generated function without iterative repair or external validation. This tests unvalidated creation.
- (3) **EvoSkill-style.** A strategy-level adaptation inspired by EvoSkill. It maintains natural-language skill hints rather than executable tools.
- (4) **ToolMaker-style.** A protocol adaptation inspired by ToolMaker’s diagnose-and-rewrite loop. It is not a native port; the original setting assumes reference tests and repository inputs that our harness does not provide.
- (5) **CREATOR-style.** A decision/execution split inspired by CREATOR, where the system explicitly decides whether a tool should be created before executing the task.

All verified-subset runs use Claude Haiku 4.5 (snapshot `claude-haiku-4-5-20251001`) via the Anthropic API with temperature 1.0 and the model default top-*p*. Each protocol is executed with three integer seeds (0, 1, 2) over eight verified-subset sessions, producing 24 session rows per protocol. Synthesised tools execute in an in-process sandbox with a five-second wall-clock limit; harness state resets between sessions so library accumulation is per-session, not cross-session. The same Anthropic API key, decoding parameters, task stream, and verifier set are used for all protocols;

only the tool-creation control flow varies. We scope this pilot to a single model snapshot on purpose: varying the model would conflate protocol effects with model-evolution effects – the latter is a separate evaluation question the same harness is built to ask, and we surface it in Supplement §E as a same-schema-different-model probe.

Pre-registration. The three contrasts in Table 4, the bootstrap parameters ($B=10,000$, seed 20260604), the BH correction, and the verified-subset definition were specified before running the Haiku pilot, and are stored in `claims.json` and `statistical_report.json` in the released artifact. No contrasts were added, removed, or redirected after results were observed. The decision to report verifier coverage, reuse decomposition, and audit traces was part of the framework design, not a response to the TC outcome.

Statistical procedure. For each pre-specified contrast we form per-session paired differences across the 24 verified-subset session rows (3 seeds $\times$ 8 sessions), bootstrap-resample those pairs $B=10,000$ times with seed 20260604, and report the bootstrap point estimate and the *nominal* (unadjusted) 95% percentile CI. Two-sided bootstrap p -values for the three contrasts are Benjamini–Hochberg adjusted at $\alpha=0.05$ and released in `claims.json`. A contrast is marked *supported* only if its CI lies strictly above zero in the pre-specified positive direction. Per-session outcomes are largely seed-stable in this pilot, so the effective number of independent units sits between 8 and 24; the bootstrap resamples (seed, session) pairs without assuming seed-independence.

Artifacts. The benchmark sources, harness, verifier definitions, canonical manifests (`run_manifest.jsonl`, `tool_manifest.jsonl`, `aggregate.json`, `audit_report.json`, `statistical_report.json`, `claims.json`), the per-tool source-preservation patch (`scripts/run_strict.py`), the held-out conformance replay (`scripts/conformance_in_regime.py`), the conformance manifests produced by the pilot (`conformance_manifest.jsonl`, `conformance_aggregate.json`), the hand-written reference implementations and their calibration check (`baselines/reference/`, `scripts/reference_conformance.py`), and the scripts that regenerate every table in this paper from disk are included in the anonymised supplement and will be released under an open licence on acceptance.

5 Results

5.1 TC alone does not separate the protocols

Verified-subset TC differences are small relative to per-session standard errors and cross-session heterogeneity. Across the five protocols TC ranges from 0.287 (CREATOR-style) to 0.358 (One-Shot), with No-Evolution (0.337) and EvoSkill-style (0.332) in between (Table 3).

Table 3: Strict verified-subset pilot. TC is computed only on tasks with deterministic verifiers; unverified tasks are reported separately and excluded from the TC denominator. Means and standard errors are over 24 session rows (3 seeds $\times$ 8 verified-subset sessions) using Claude Haiku 4.5.

System	TC	SE	Tools	Correct-reuse rate
One-Shot	0.358	0.077	114	0.333
No-Evolution	0.337	0.064	0	0.333
EvoSkill-style	0.332	0.062	0	0.250
ToolMaker-style	0.292	0.054	18	0.375
CREATOR-style	0.287	0.048	111	0.333

Table 4: Pre-specified TC contrasts in the strict verified-subset pilot. Positive effects were expected; none support a TC-improvement claim. Values are absolute TC differences.

Contrast	Δ	95% CI	Improves?
OneShot – NoEvol	0.0208	[−0.0347, 0.0729]	No
Creator – OneShot	−0.0710	[−0.1389, −0.0012]	No
ToolMaker – OneShot	−0.0655	[−0.1443, 0.0174]	No

The pre-specified improvement tests do not support a positive TC claim (Table 4). One-Shot is not distinguishable from No-Evolution in the expected positive direction. ToolMaker-style does not improve over One-Shot. The Creator – One-Shot contrast is the only pre-specified comparison whose nominal 95% CI excludes zero: −7.1 pp with CI [−13.9, −0.1], just brushing zero from below (two-sided bootstrap $p=0.044$). Under the pre-registered BH correction across the three contrasts the adjusted p is 0.133, so the contrast does not survive multiplicity adjustment – consistent with the hypothesis-generating reading we give it. The CI is consistent with – but does not establish – a synthesis-loop cost in the adapted-baseline regime. We do not draw a method claim about the original CREATOR design from an adapted protocol; we report this only as a signal worth a powered follow-up.

Power. With 24 paired session rows and the observed within-session standard deviation of TC (≈ 0.30), and accounting for the seed-stability above, the verified-subset pilot is powered (80%, $\alpha=0.05$) to detect paired TC differences of roughly 8–13 pp depending on whether effective n is treated as 24 or as 8 sessions. The largest observed $|\Delta|$ in Table 4 is 7.1 pp. The null TC result should therefore be read as “no large protocol effect at this scale” rather than “no effect.” Smaller true effects – below ~ 7 pp – remain compatible with these CIs and motivate the larger multi-model pilots that the released harness is built to support.

5.2 The audit view separates what TC does not

The lack of a TC winner is precisely when the audit layer matters most. One-Shot and CREATOR-style create many tools (114 and 111 respectively), while No-Evolution and EvoSkill-style create none. ToolMaker-style creates fewer tools (18), consistent with a more conservative validation loop. These differences are invisible in a TC-only table. The reuse denominators in Table 3 are non-zero even for the zero-tools systems: reuse counts include calls to provided seed tools (the seed library each session starts from), not only to tools the agent itself created.

The correct-reuse rate – the fraction of reuse calls that coincide with a passing task – adds a second axis. One-Shot records 18 correct and 36 incorrect reuses (rate 0.33), No-Evolution 9 correct and 18 incorrect (0.33), EvoSkill-style 9 correct and 27 incorrect (0.25), CREATOR-style 12 correct and 24 incorrect (0.33), and ToolMaker-style 9 correct and 15 incorrect (0.38). All counts are multiples of three because per-session reuse decisions are largely identical across the three seeds, consistent with the seed-stability noted in §4. At this scale the differences are not powered to discriminate between protocols, and we report them only for trace inspection and protocol design. What they show is the kind of decomposition the harness exposes: when a system uses prior artifacts, does that reuse coincide with success or with failure?

5.3 A verification-vs.-conformance gap that TC cannot see

A tool can pass every in-session verifier call and still fail every held-out input for the same capability – and TC will not see it. The verifier asks whether one particular call produced an acceptable output; a capability-aligned conformance suite asks whether the tool implements the capability on inputs the agent never saw. These two questions routinely disagree: a tool that structurally matches the example the agent used to write itself will pass the verifier and fail held-out conformance. That gap is what motivates the per-tool conformance layer the framework defines, and it is the failure mode an audit schema for a deployed tool-evolving agent has to surface.

In-regime evidence on the Haiku pilot. To demonstrate the gap in the same regime as Tables 3–4, we patched the strict harness to preserve per-tool source and replayed every synthesised tool against the held-out conformance suite for the capability it was created for (five to six hidden unit inputs plus five to six adversarial probes per capability; tools whose creating task links multiple capabilities are scored against the union, so per-tool suite sizes range from 10 to 22 inputs, median 11). Because the runs behind Table 3 predate the source-preservation patch, the probe is a *separate re-execution* of the same matrix – same model snapshot, seeds, sessions, and verifiers, at temperature 1.0 – so creation counts differ stochastically from Table 3; session-level TC in the re-execution (0.31–0.34 across the three systems) is consistent with Table 3. Table 5 counts *distinct preserved tools* (final

Table 5: In-regime held-out conformance on the Haiku 4.5 verified-subset pilot (separate re-execution of the Table 3 matrix with per-tool source preservation; see §5.3). Each distinct preserved tool is replayed against the held-out conformance suite (10–22 inputs, median 11) the agent never saw. “Silent rot %” is the fraction of preserved tools with held-out correctness $C=0.00$. Mean C and mean R (robustness: executes without crashing on adversarial inputs) are across all preserved tools.

System	Tools	Mean C	Mean R	Silent rot %
One-Shot	119	0.000	0.518	100.0%
ToolMaker-style	19	0.000	0.526	100.0%
CREATOR-style	84	0.049	0.536	91.7%
Total	222	0.018	0.525	96.8%

source per tool name per session and seed); CREATOR-style’s rewrite loop produces 105 creation events that collapse to 84 distinct tools. We say a preserved tool exhibits *silent rot* if it executes and passed its in-session verifier calls but records $C=0.00$ on the held-out suite. The pattern is near-universal: *none* of One-Shot’s 119 or ToolMaker-style’s 19 tools have non-zero held-out correctness; of CREATOR-style’s 84 tools, 7 (8.3%) achieve a non-zero score (one reaches $C=1.00$). The remaining tools execute cleanly – mean held-out robustness is 0.52–0.54 in all three systems (Table 5) – they simply return wrong answers on inputs the agent never saw. The verification-vs.-conformance gap is therefore not an isolated trace but a systematic property of unvalidated synthesis at Haiku scale, and exactly the failure mode an audit schema for a deployed tool-evolving agent has to surface.

Calibration: the 96.8% is a discriminator, not a floor. A natural concern is that conformance suites might simply be too hard. Four lines of evidence rule that out. (i) Tests are authored alongside the gap task that introduces a capability, before any tool exists; they are not adversarial against any specific synthesis. (ii) Hand-written reference implementations, released with the benchmark, score $C=1.00$ on *all* 16 capability suites (`scripts/reference_conformance.py`); this audit also caught two suite entries that were unpassable by construction (missing oracles), which we repaired – no synthesised tool’s score changed under the repair. (iii) The suites are passable by the same model in the same regime: 7 of 84 CREATOR-style tools (8.3%) achieve non-zero held-out correctness, and one capability (`vd1_record_validate`, synthesised in `data_transform.s2/gap_2`) reaches $C=1.00$ on all three seeds independently – same model, same harness, same conformance machinery. (iv) Tools are syntactically valid: $C=0.00$ means *wrong on every held-out input*, not *crashed on every held-out input*. Mean held-out robustness across the 222 tools is 0.52–0.54 in all three systems (Table 5), so silent-rot tools execute cleanly and return values; the values are simply incorrect for the capability they claim to implement.

Two anchored examples (same regime). The contrast between a silent-rot tool and a working tool is sharp. CREATOR-style on `data.transform.s1/gap.1` synthesises `decode.arise.binary.records` for the `abr.binary.decode` capability; the tool passes in-session reuse calls but its held-out conformance is $C=0.00$ across the capability’s 12 inputs the agent never saw ($R=0.50$, code-quality 0.78 – it executes cleanly on half of inputs and returns wrong values). The same protocol on the next session synthesises `validate.records.against.schema` for the `vd1.record.validate` capability and hits $C=1.00$ on that capability’s 11 held-out inputs ($R=0.83$, generality 1.00), reproduced identically across seeds 0, 1, and 2 (suite sizes vary slightly by capability, here 12 vs. 11). Same model snapshot, same harness, same conformance pipeline. The framework discriminates working tool synthesis from silent rot at the per-tool level.

Why $C=0.00$ coexists with unchanged TC. If 96.8% of synthesised tools are wrong on held-out inputs, why is One-Shot’s TC the *highest* in Table 3? Because the two measurements see different input distributions. The in-session verifier scores the one call the task needed – an input structurally within the regime the tool was just fit to – so an overfit tool passes it; the conformance suite draws from the capability’s full contract, which the tool never saw. A library of $C=0.00$ tools therefore leaves TC untouched as long as tasks stay inside the fitting regime, and the reuse decomposition in §5.2 shows the in-session shadow of the same defect: roughly two-thirds of reuse calls coincide with task failure in every tool-creating protocol. This is not a contradiction in the data; it is the deployment risk itself. TC is computed on the distribution the agent saw, conformance on the distribution the next release will see.

Cross-model supporting trace. A preserved-source Sonnet pilot reproduces the same pattern at session granularity (Supplement §D): one tool, `decode.abr.format` for the `abr.binary.decode` capability, drives session TC to 0.95 while recording $C=0.00$ on the same held-out conformance suite. The pattern – in-session verifier calls satisfied, every held-out input failed – holds across the model swap.

5.4 Verifier coverage is a first-class benchmark property

If unverified tasks are credited by a length or plausibility heuristic, absolute TC can be inflated and protocol comparisons move for the wrong reason. Restricting the main analysis to deterministic verifiers sacrifices benchmark coverage but buys measurement credibility. The cost is clear: the API-orchestration domain is not part of the verified-subset TC analysis. The framework already identifies what to do next: complete API verifiers, align hidden tests by capability, and regenerate the same tables from the full manifest. Adversarial probes are part of the design (Table 2 reports 56.2% coverage). The current pilot records pass/fail at session level rather than per role, so per-records adversarial pass rates are

not isolated in this submission; closing that gap is part of the same coverage-and-trace work as the API verifiers.

6 Discussion

Design lessons for tool-evolving agents. Tool creation alone is not evidence of progress: many tools can be created without improving verified task completion. Reuse needs a quality signal; raw reuse is ambiguous without correct/incorrect decomposition. Verifier coverage is a first-class benchmark property – a tool-evolving system is only as trustworthy as the tests and records used to decide what it has learned. Generated tools are persistent objects that can be reused, duplicated, retired, or accidentally made into load-bearing dependencies for later failures; a benchmark that ignores this treats every run as independent and loses the audit trail the deployment setting needs.

Model evolution as audit signal. The same harness, manifests, and verifier set apply equally to a Sonnet pilot reported in Supplement §E, where per-tool source preservation made the capability-aligned conformance layer computable. Library-Health contrasts that reach statistical support in that supplementary Sonnet pilot – supporting evidence, not a main-paper claim – are not powered under Haiku, not because the framework fails but because synthesis quality at the smaller model is below the floor where reuse decomposition becomes informative. The audit schema thus doubles as a model-evolution monitor: re-emitting the same schema after a model swap reveals whether tool quality has tracked the model, and the comparison is the exact “model evolution and API risk” question that motivates monitoring deployed agents.

From benchmark to post-deployment monitor. We see the audit schema as a CI artifact, not only a research instrument. A tool-evolving agent shipped behind an API can re-emit `run.manifest.jsonl` and `tool.manifest.jsonl` on every release; a regression gate can then require that the correct-reuse rate and capability-aligned hidden-test conformance do not drop below the prior release on a frozen task slice. Per-task audit overhead is dominated by the LLM calls themselves – manifest emission, reuse decomposition, and per-tool source dumping add millisecond-scale latency and bounded disk I/O, so the layer is cheap enough to leave on in production. Concretely, three monitoring signals fall out of the schema with no extra instrumentation: (i) *silent rot* – session TC stays high while held-out conformance for a re-used capability drops between releases; (ii) *reuse trap* – raw reuse rises but the correct-reuse rate falls, indicating an emerging defective dependency; (iii) *regression delta* – a regress-role task fails after the same capability’s gap-role task passes, identifying which release crossed the regression. Each signal is a thresholded check that can sit behind a release gate.

7 Limitations

Verified subset. The strict statistical analysis covers 8 sessions and 51 verified task decisions per system pass, not the full 99-task benchmark. The API-orchestration session is excluded from main TC until deterministic verifiers are complete.

Pilot scale and single-model setting. The verified-subset pilot uses one model (Claude Haiku 4.5) and three integer seeds; per-session outcomes are partly seed-stable so standard errors mainly reflect session heterogeneity. A smaller model is where tool-creation protocols are most stress-tested, so the null TC result is consistent with synthesis quality at Haiku scale being below the floor where protocol differences become powered. The pilot should be read as exposing what the framework decomposes and what minimum effect size it can rule out (§5), not as a verdict on tool-creation protocols in general. Multi-model runs are the natural next step the released harness is built for, and are the lens through which the model-evolution monitoring story in §6 can be operationalized.

Protocol adaptations. EvoSkill-style, ToolMaker-style, and CREATOR-style are adaptations to a shared open-task harness, not native reproductions of the original systems. The original CREATOR and ToolMaker pipelines assume reference test suites or repository-grounded inputs that our open synthesis tasks do not supply; the `-style` suffix is deliberate, and the paper does not treat any underperformance in this pilot as a verdict on the original methods.

In-regime conformance scope. The in-regime conformance probe in §5.3 covers the three tool-creating protocols on the verified-subset Haiku matrix ($n=222$ preserved tools across $3 \text{ seeds} \times 8 \text{ sessions}$ per system). The probe is single-model: held-out correctness at Sonnet or larger models could change the silent-rot rates reported in Table 5, and the released harness is built to ask that question without re-running the protocols.

Composite library-health score. A composite library-health score (or *EvolveTool-Score*) is useful as a dashboard but premature until conformance and verifier maps are complete; the current submission reports decomposed metrics.

8 Conclusion

Tool-evolving agents create a new evaluation target: the library they leave behind. A task-completion score is necessary but not sufficient. EVOLVETOOL-BENCH provides a structured way to inspect this artifact through task roles, verifier-aware TC, reuse decomposition, capability-aligned conformance, and trace preservation. The verified-subset pilot does not support a TC-improvement claim for any tested protocol, but it surfaces an in-regime verification-vs.-conformance gap that is systematic: 96.8% of 222 preserved tools across three tool-creating protocols record $C=0.00$ on the held-out conformance suite for the capability they were synthesised for – suites that hand-written reference implementations pass at $C=1.00$ – while still passing in-session verifier calls and executing without errors. The same audit schema surfaces

three thresholded monitoring signals – silent rot, reuse trap, regression delta – that a deployed tool-evolving agent can re-emit between releases.

Reusable Evaluation Practices

We recommend that future benchmarks for tool-evolving agents:

- (1) separate task success from generated-artifact behavior;
- (2) report verifier coverage and exclude unverified tasks from TC claims;
- (3) distinguish correct reuse from incorrect reuse;
- (4) preserve generated artifacts, artifact summaries, and task/session traces;
- (5) include regression and composition probes after library growth;
- (6) evaluate library-management policies such as promotion, deduplication, and retirement.

References

- [Alzubi et al.(2026)] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyan Chen, and Tu Vu. 2026. EvoSkill: Automated Skill Discovery for Multi-Agent Systems. *arXiv preprint arXiv:2603.02766* (2026).
- [Backlund and Petersson(2025)] Axel Backlund and Lukas Petersson. 2025. Vending-Bench: A Benchmark for Long-Term Coherence of Autonomous Agents. *arXiv preprint arXiv:2502.15840* (2025).
- [Cai et al.(2023)] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2023. Large Language Models as Tool Makers. *arXiv preprint arXiv:2305.17126* (2023).
- [Ellis et al.(2021)] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2021. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*.
- [Li et al.(2026)] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).
- [Liu et al.(2024)] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuananyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.
- [Qian et al.(2023)] Cheng Qian, Chi Han, Yi R. Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.
- [Qin et al.(2023)] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).
- [Singhal et al.(2024)] Manav Singhal, Tushar Aggarwal, Abhijeet Awasthi, Nagarajan Natarajan, and Aditya Kanade. 2024. No-FunEval: Funny How Code LMs Falter on Requirements Beyond Functional Correctness. *arXiv preprint arXiv:2401.15963* (2024).
- [Wang et al.(2023)] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Advances in Neural Information Processing Systems* (2023).
- [Wölflin et al.(2025)] Georg Wölflin, Dyke Ferber, Daniel Truhn, Ognjen Arandjelović, and Jakob Nikolas Kather. 2025. LLM Agents Making Agent Tools. *arXiv preprint arXiv:2502.11705* (2025).
- [Xia et al.(2026)] Bowei Xia, Mengkang Hu, Shijian Wang, Jiarui Jin, Wenxiang Jiao, Yuan Lu, Kexin Li, and Ping Luo. 2026.

813	Tool-Genesis: A Task-Driven Tool Creation Benchmark for Self-Evolving Language Agents. <i>arXiv preprint arXiv:2603.05578</i> (2026).	[Zhang et al.(2026)] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, et al. 2026. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. <i>arXiv preprint arXiv:2604.01687</i> (2026).	871
814			872
815			873
816			874
817			875
818			876
819			877
820			878
821			879
822			880
823			881
824			882
825			883
826			884
827			885
828			886
829			887
830			888
831			889
832			890
833			891
834			892
835			893
836			894
837			895
838			896
839			897
840			898
841			899
842			900
843			901
844			902
845			903
846			904
847			905
848			906
849			907
850			908
851			909
852			910
853			911
854			912
855			913
856			914
857			915
858			916
859			917
860			918
861			919
862			920
863			921
864			922
865			923
866			924
867			925
868			926
869			927
870			928